

When to Finetune

Before jumping into different finetuning techniques, it's necessary to consider whether finetuning is the right option for you. Compared to prompt-based methods, finetuning requires significantly more resources, not just in data and hardware, but also in ML talent. Therefore, finetuning is generally attempted *after* extensive experiments with prompt-based methods. However, finetuning and prompting aren't mutually exclusive. Real-world problems often require both approaches.

Reasons to Finetune

The primary reason for finetuning is to improve a model's quality, in terms of both general capabilities and task-specific capabilities. Finetuning is commonly used to improve a model's ability to generate outputs following specific structures, such as JSON or YAML formats.

A general-purpose model that performs well on a wide range of benchmarks might not perform well on your specific task. If the model you want to use wasn't sufficiently trained on your task, finetuning it with your data can be especially useful.

For example, an out-of-the-box model might be good at converting from text to the standard SQL dialect but might fail with a less common SQL dialect. In this case, finetuning this model on data containing this SQL dialect will help. Similarly, if the model works well on standard SQL for common queries but often fails for customer-specific queries, finetuning the model on customer-specific queries might help.

One especially interesting use case of finetuning is bias mitigation. The idea is that if the base model perpetuates certain biases from its training data, exposing it to carefully curated data during finetuning can counteract these biases (Wang and Russakovsky, 2023). For example, if a model consistently assigns CEOs male-sounding names, finetuning it on a dataset with many female CEOs can mitigate this bias. Gari-mella et al. (2022) found that finetuning BERT-like language models on text authored by women can reduce these models' gender biases, while finetuning them on texts by African authors can reduce racial biases.

You can finetune a big model to make it even better, but finetuning smaller models is much more common. Smaller models require less memory, and, therefore, are easier to finetune. They are also cheaper and faster to use in production.

A common approach is to finetune a small model to imitate the behavior of a larger model using data generated by this large model. Because this approach distills the larger model's knowledge into the smaller model, it's called *distillation*. This is discussed in Chapter 8 together with other data synthesis techniques.

A small model, finetuned on a specific task, might outperform a much larger out-of-the-box model on that task. For example, Grammarly found that their finetuned Flan-T5 models (Chung et al., 2022) outperformed a GPT-3 variant specialized in text editing across a wide range of writing assistant tasks despite being 60 times smaller. The finetuning process used only 82,000 (instruction, output) pairs, which is smaller than the data typically needed to train a text-editing model from scratch.

In the early days of foundation models, when the strongest models were commercial with limited finetuning access, there weren't many competitive models available for finetuning. However, as the open source community proliferates with high-quality models of all sizes, tailored for a wide variety of domains, finetuning has become a lot more viable and attractive.

Reasons Not to Finetune

While finetuning can improve a model in many ways, many of these improvements can also be achieved, to a certain extent, without finetuning. Finetuning can improve a model's performance, but so do carefully crafted prompts and context. Finetuning can help with structured outputs, but many other techniques, as discussed in Chapter 2, can also do that.

First, while finetuning a model for a specific task can improve its performance for that task, it can degrade its performance for other tasks.¹ This can be frustrating when you intend this model for an application that expects diverse prompts.

¹ Some people call this phenomenon an alignment tax (Bai et al., 2020), but this term can be confused with penalties against human preference alignment.

Imagine you need a model for three types of queries: product recommendations, changing orders, and general feedback. Originally, the model works well for product recommendations and general feedback but poorly for changing orders. To fix this, you finetune the model on a dataset of (query, response) pairs about changing orders. The finetuned model might indeed perform better for this type of query, but worse for the two other tasks.

What do you do in this situation? You can finetune the model on all the queries you care about, not just changing orders. If you can't seem to get a model to perform well on all your tasks, consider using separate models for different tasks. If you wish to combine these separate models into one to make serving them easier, you can also consider merging them together, as discussed later in this chapter.

If you're just starting to experiment with a project, finetuning is rarely the first thing you should attempt. Finetuning requires high up-front investments and continual maintenance. First, you need data. Annotated data can be slow and expensive to acquire manually, especially for tasks that demand critical thinking and domain expertise. Open source data and AI-generated data can mitigate the cost, but their effectiveness is highly variable.

Second, finetuning requires the knowledge of how to train models. You need to evaluate base models to choose one to finetune. Depending on your needs and resources, options might be limited. While finetuning frameworks and APIs can automate many steps in the actual finetuning process, you still need to understand the different training knobs you can tweak, monitor the learning process, and debug when something is wrong. For example, you need to understand how an optimizer works, what learning rate to use, how much training data is needed, how to address overfitting/underfitting, and how to evaluate your models throughout the process.

Third, once you have a finetuned model, you'll need to figure out how to serve it. Will you host it yourself or use an API service? As discussed in [Chapter 9](#), inference optimization for large models, especially LLMs, isn't trivial. Finetuning requires less of a technical leap if you're already hosting your models in-house and familiar with how to operate models.

More importantly, you need to establish a policy and budget for monitoring, maintaining, and updating your model. As you iterate on your finetuned model, new base models are being developed at a rapid pace. These base models may improve faster than you can enhance your finetuned model. If a new base model outperforms your finetuned model on your specific task, how significant does the performance improvement have to be before you switch to the new base model? What if a new base model doesn't immediately outperform your existing model but has the potential to do so after finetuning—would you experiment with it?

In many cases, switching to a better model would provide only a small incremental improvement, and your task might be given a lower priority than projects with larger returns, like enabling new use cases.²

AI engineering experiments should start with prompting, following the best practices discussed in [Chapter 6](#). Explore more advanced solutions only if prompting alone proves inadequate. Ensure you have thoroughly tested various prompts, as a model's performance can vary greatly with different prompts.

Many practitioners I've spoken with share a similar story that goes like this. Someone complains that prompting is ineffective and insists on finetuning. Upon investigation, it turns out that prompt experiments were minimal and unsystematic. Instructions were unclear, examples didn't represent actual data, and metrics were poorly defined. After refining the prompt experiment process, the prompt quality improved enough to be sufficient for their application.³

Finetuning Domain-Specific Tasks

Beware of the argument that general-purpose models don't work well for domain-specific tasks, and, therefore, you must finetune or train models for your specific tasks. As general-purpose models become more capable, they also become better at domain-specific tasks and can outperform the domain-specific models.

An interesting early specialized model is BloombergGPT, which was introduced by Bloomberg in March 2023. The strongest models on the market then were all proprietary, and Bloomberg wanted a mid-size model that performed well on financial tasks and could be hosted in-house for use cases with sensitive data. The model, with 50 billion parameters, required 1.3 million A100 GPU hours for training. The estimated cost of the compute was between \$1.3 million and \$2.6 million, excluding data costs ([Wu et al., 2023](#)).

In the same month, OpenAI released GPT-4-0314.⁴ Research by [Li et al. \(2023\)](#) demonstrated that GPT-4-0314 significantly outperformed BloombergGPT across various financial benchmarks. [Table 7-1](#) provides details of two such benchmarks.

2 Many businesses resist changing technologies they consider "good enough." If all companies were quick to adopt more optimal solutions, fax machines would have become obsolete by now.

3 I've also noticed a few cases when engineers know that finetuning isn't strictly necessary but still insist on doing it because they want to learn how to finetune. As an engineer who likes learning new skills, I appreciate this mindset. However, if you're in a leadership position, it can be hard to differentiate whether finetuning is needed or wanted.

4 0314 denotes the date this GPT-4 version came out, March 14, 2024. The specific date stamp matters because different versions vary significantly in performance.

Table 7-1. General-purpose models like GPT-4 can outperform financial models in financial domains.

Model	FiQA sentiment analysis (weighted F1)	ConvFinQA (accuracy)
GPT-4-0314 (zero-shot)	87.15	76.48
BloombergGPT	75.07	43.41

Since then, several mid-size models with performance comparable to GPT-4 have been released, including **Claude 3.5 Sonnet** (70B parameters), **Llama 3-70B-Instruct**, and **Qwen2-72B-Instruct**. The latter two are open weight and can be self-hosted.

Because benchmarks are insufficient to capture real-world performance, it's possible that BloombergGPT works well for Bloomberg for their specific use cases. The Bloomberg team certainly gained invaluable experience through training this model, which might enable them to better develop and operate future models.

Both finetuning and prompting experiments require systematic processes. Doing prompt experiments enables developers to build an evaluation pipeline, data annotation guideline, and experiment tracking practices that will be stepping stones for finetuning.

One benefit of finetuning, before prompt caching was introduced, was that it can help optimize token usage. The more examples you add to a prompt, the more input tokens the model will use, which increases both latency and cost. Instead of including your examples in each prompt, you can finetune a model on these examples. This allows you to use shorter prompts with the finetuned model, as shown in **Figure 7-2**.

With prompt caching, where repetitive prompt segments can be cached for reuse, this is no longer a strong benefit. Prompt caching is discussed further in **Chapter 9**. However, the number of examples you can use with a prompt is still limited by the maximum context length. With finetuning, there's no limit to how many examples you can use.

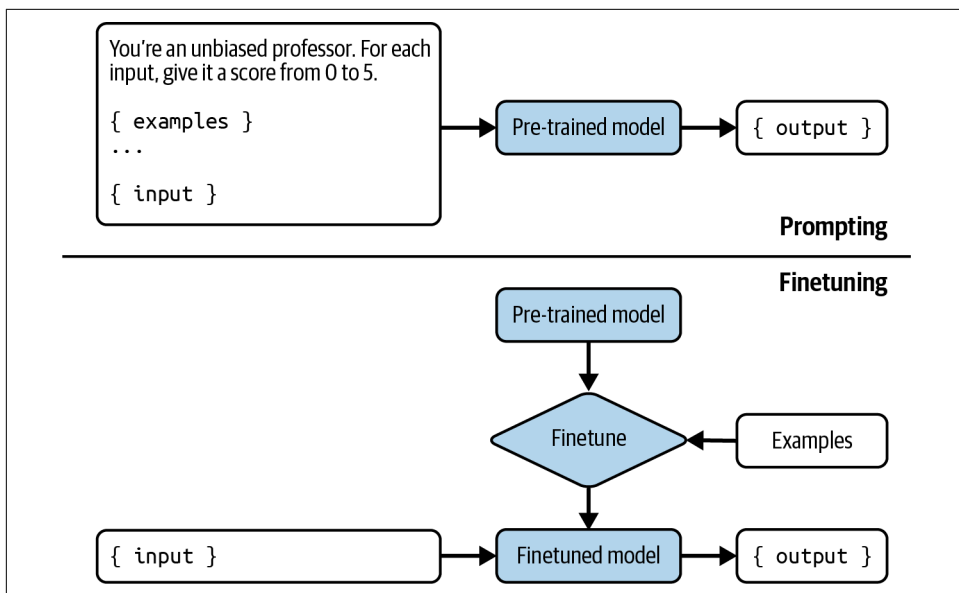


Figure 7-2. Instead of including examples in each prompt, which increases cost and latency, you finetune a model on these examples.

Finetuning and RAG

Once you've maximized the performance gains from prompting, you might wonder whether to do RAG or finetuning next. The answer depends on whether your model's failures are information-based or behavior-based.

If the model fails because it lacks information, a RAG system that gives the model access to the relevant sources of information can help. Information-based failures happen when the outputs are factually wrong or outdated. Here are two example scenarios in which information-based failures happen:

The model doesn't have the information.

Public models are unlikely to have information private to you or your organization. When a model doesn't have the information, it either tells you so or hallucinates an answer.

The model has outdated information.

If you ask: "How many studio albums has Taylor Swift released?" and the correct answer is 11, but the model answers 10, it can be because the model's cut-off date was before the release of the latest album.

The paper "[Fine-Tuning or Retrieval?](#)" by Ovadia et al. (2024) demonstrated that for tasks that require up-to-date information, such as questions about current events, RAG outperformed finetuned models. Not only that, RAG with the base model

outperformed RAG with finetuned models, as shown in [Table 7-2](#). This finding indicates that *while finetuning can enhance a model’s performance on a specific task, it may also lead to a decline in performance in other areas*.

Table 7-2. RAG outperforms finetuning on a question-answering task about current events, curated by Ovadia et al. (2024). FT-reg and FT-par refer to two different finetuning approaches the author used.

	Base model	Base model + RAG	FT-reg	FT-par	FT-reg + RAG	FT-par + RAG
Mistral-7B	0.481	0.875	0.504	0.588	0.810	0.830
Llama 2-7B	0.353	0.585	0.219	0.392	0.326	0.520
Orca 2-7B	0.456	0.876	0.511	0.566	0.820	0.826

On the other hand, *if the model has behavioral issues, finetuning might help*. One behavioral issue is when the model’s outputs are factually correct but irrelevant to the task. For example, you ask the model to generate technical specifications for a software project to provide to your engineering teams. While accurate, the generated specs lack the details your teams need. Finetuning the model with well-defined technical specifications can make the outputs more relevant.

Another issue is when it fails to follow the expected output format. For example, if you asked the model to write HTML code, but the generated code didn’t compile, it might be because the model wasn’t sufficiently exposed to HTML in its training data. You can correct this by exposing the model to more HTML code during finetuning.

Semantic parsing is a category of tasks whose success hinges on the model’s ability to generate outputs in the expected format and, therefore, often requires finetuning. Semantic parsing is discussed briefly in [Chapters 2](#) and [6](#). As a reminder, semantic parsing means converting natural language into a structured format like JSON. Strong off-the-shelf models are generally good for common, less complex syntaxes like JSON, YAML, and regex. However, they might not be as good for syntaxes with fewer available examples on the internet, such as a domain-specific language for a less popular tool or a complex syntax.

In short, finetuning is for form, and RAG is for facts. A RAG system gives your model external knowledge to construct more accurate and informative answers. A RAG system can help mitigate your model’s hallucinations. Finetuning, on the other hand, helps your model understand and follow syntaxes and styles.⁵ While finetuning can potentially reduce hallucinations if done with enough high-quality data, it can also worsen hallucinations if the data quality is low.

⁵ Some people, such as the authors of the Llama 3.1 paper ([Dubey et al., 2024](#)), adhere to “the principle that post-training should align the model to ‘know what it knows’ rather than add knowledge.”

If your model has both information and behavior issues, start with RAG. RAG is typically easier since you won't have to worry about curating training data or hosting the finetuned models. When doing RAG, start with simple term-based solutions such as BM25 instead of jumping straight into something that requires vector databases.

RAG can also introduce a more significant performance boost than finetuning. Ovardia et al. (2024) showed that for almost all question categories in the **MMLU benchmark**, RAG outperforms finetuning for three different models: Mistral 7B, Llama 2-7B, and Orca 2-7B.

However, RAG and finetuning aren't mutually exclusive. They can sometimes be used together to maximize your application's performance. In the same experiment, Ovardia et al. (2024) showed that incorporating RAG on top of a finetuned model can boost its performance on the MMLU benchmark 43% of the time. It's important to note that in this experiment, using RAG with finetuned models doesn't improve the performance 57% of the time, compared to using RAG alone.

There's no universal workflow for all applications. **Figure 7-3** shows some paths an application development process might follow over time. The arrow indicates what next step you might try. This figure is inspired by an example workflow shown by OpenAI (2023).

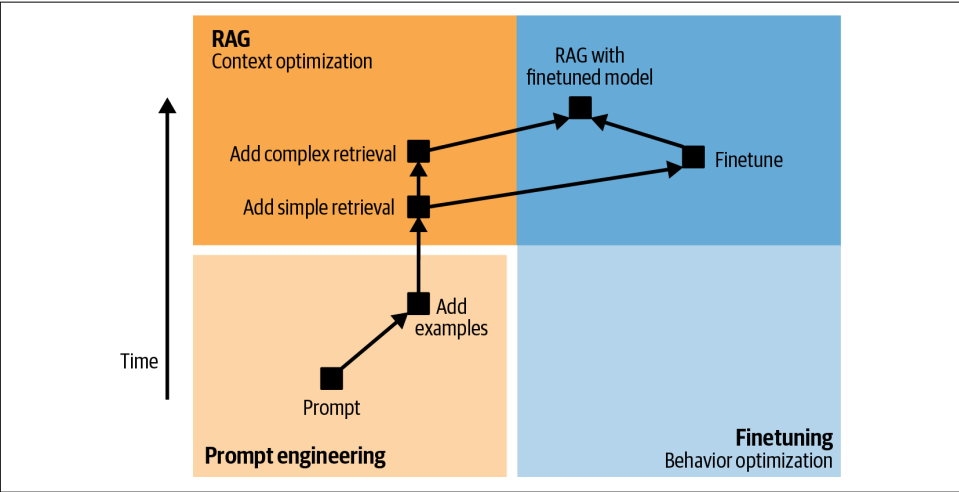


Figure 7-3. Example application development flows. After simple retrieval (such as term-based retrieval), whether to experiment with more complex retrieval (such as hybrid search) or finetuning depends on each application and its failure modes.

So the workflow to adapt a model to a task might work as follows. Note that before any of the adaptation steps, you should define your evaluation criteria and design your evaluation pipeline, as discussed in **Chapter 4**. This evaluation pipeline is what you'll use to benchmark your progress as you develop your application. Evaluation